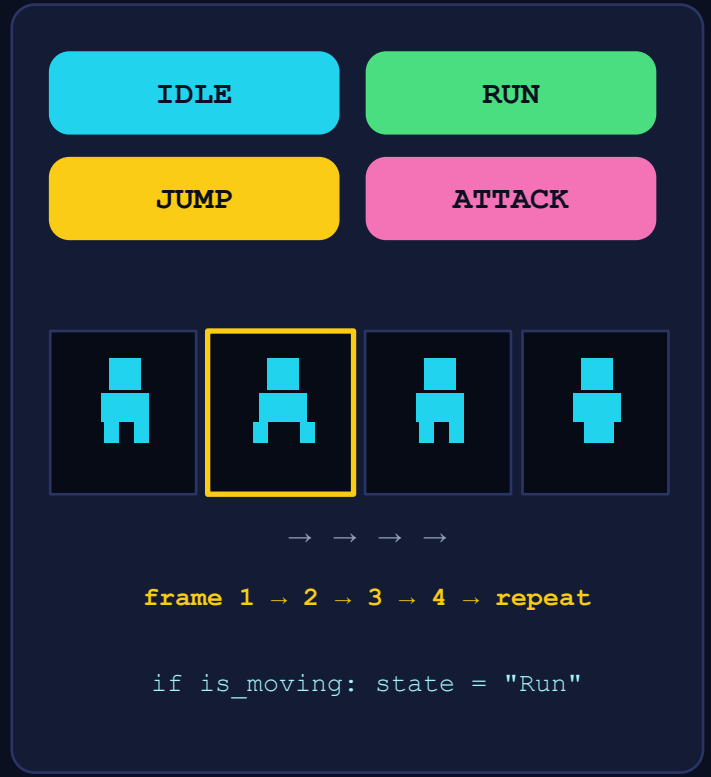
CCA GAME DEVELOPMENT III

State-Machine Animation

Creating Dynamic Character Animations with Python & Pygame

Module 3 — Session 10

Teacher: Annisa Hendrawan
Senior High School - Global Prestasi School



Quick Review: Sprite Sheets & JSON

What did we learn in the previous session?

- A sprite sheet contains multiple images of a character.
- Each image represents one animation frame.
- Coordinates describe the position and size of each frame.
- JSON stores sprite information in a structured format.
- Python can read JSON data using the `json` module.



REMEMBER

A sprite sheet stores the **images**, while JSON stores information about **where each frame is located**.

Quick Question: **If a sprite sheet contains 6 character images, how many frames can we extract from it?**

From Sprite Frames to Animation

How do we make a character move? An animation is created by displaying multiple frames in sequence.



Frame 1
Left leg forward.



Frame 2
Legs in the middle position.



Frame 3
Right leg forward.



Frame 4
Legs return to the first position.

When these frames are displayed repeatedly, the character appears to run.

Frame
One individual image.

Animation
A sequence of frames.

FPS (Frames Per Second)
The number of frames displayed per second.

Quick Question: What happens if the frames change too slowly?

Understanding Animation Speed

How do we control animation speed? We use FPS to determine how quickly animation frames change.

FORMULA 1

Frame Duration

```
Frame Duration = 1 / FPS
```

Example: If the animation runs at 5 FPS:

1 / 5 = 0.2 seconds

Each frame is displayed for 0.2 seconds.

FORMULA 2

Animation Duration

```
Duration = Total Frames / FPS
```

Example: 6 frames at 6 FPS:

6 / 6 = 1 second

One complete animation cycle takes 1 second.



Higher FPS = Faster animation.

Lower FPS = Slower animation.

Displaying Animation Frames in Pygame

Let's make the frames change automatically!

First, we need a list containing the animation images.

```
idle_frames = [
    pygame.image.load("idle_1.png"),
    pygame.image.load("idle_2.png"),
    pygame.image.load("idle_3.png")
]
```

Next, we need to select which frame should be displayed.



idle_frames[0..2]

FORMULA · CURRENT FRAME

```
Frame Index =
floor( Elapsed Time
      / Frame Duration )
mod Total Frames
```

The modulo (%) operator makes the animation return to the first frame after reaching the last frame.



LEVEL 05 · PYTHON EXAMPLE

Choosing the Frame to Display

Python Example:

```
frame_index = (elapsed_time // frame_duration) % len(idle_frames)
screen.blit(idle_frames[frame_index], (150, 100))
```

WHAT DO THESE COMMANDS DO?

`idle_frames`	Stores the animation frames.
`frame_index`	Selects the current frame.
`len(idle_frames)`	Counts the available frames.
`screen.blit()`	Displays the selected image.

Note: This example assumes that the individual frame images have already been prepared. Extracting frames from a sprite sheet is the previous Session 9 task.

What Is an Animation State?

Different actions need different animations. A state describes what the character is currently doing.

IDLE



Standing still

When the player is not moving, use Idle.

RUN



Moving

When the player moves, use Run.

JUMP



Jumping

When the player jumps, use Jump.

ATTACK



Attacking

When the player attacks, use Attack.

Quick Question: Should a character use the Run animation while standing still?

What Is a State Machine?

A **State Machine** is a system that controls which state is active based on the character's input and conditions.

EXAMPLE OF STATE TRANSITIONS

CURRENT STATE		CONDITION		NEXT STATE
Idle	→	Movement key pressed	→	Run
Run	→	Movement key released	→	Idle
Idle	→	Jump key pressed and grounded	→	Jump
Run	→	Jump key pressed and grounded	→	Jump
Attack	→	Attack animation finished	→	Idle



The State Machine **decides which animation should play.**

The Animation System **displays the frames of that animation.**

State Machine Using Python

We can use conditional statements to select the current state. Python uses **`if`**, **`elif`**, and **`else`** to check conditions.

```
if is_attacking:
    current_state = "Attack"
elif is_jumping:
    current_state = "Jump"
elif is_moving:
    current_state = "Run"
else:
    current_state = "Idle"
```

HOW DOES IT WORK?

1. Check if the character is attacking.
2. If not, check if the character is jumping.
3. If not, check if the character is moving.
4. If none of the conditions are true, use Idle.

Important: **The order of conditions determines state priority. For example, Attack has priority over Jump and Run in this code.**

Connecting States to Animations

How does the game choose the correct animation? Store the animation frame lists in a Python dictionary.

```
animations = {
    "Idle": idle_frames,
    "Run": run_frames,
    "Jump": jump_frames,
    "Attack": attack_frames
}
```

Then select the animation based on the current state:

```
current_animation = animations[current_state]
```

Now we can display a frame from the selected animation:

```
frame_index = (elapsed_time // frame_duration)
              % len(current_animation)
screen.blit(current_animation[frame_index], (150, 100))
```

ANIMATION PROCESS

1. Player provides input.
2. Game checks character conditions.
3. State Machine selects a state.
4. Python selects the corresponding animation.
5. Pygame displays the animation frames.

LEVEL 10 · LET'S PRACTICE!

Mini Mission: Identify the Animation State

Imagine you are creating a simple platform game. Determine the correct animation state for each situation:

SITUATION	ANIMATION STATE
Character is standing still	?
Character moves to the right	?
Character jumps	?
Character performs an attack	?
Character stops moving	?

CHALLENGE QUESTIONS

1. What happens when the player changes from Run to Jump?
2. Why does each action need a different animation?
3. What happens if the State Machine selects the wrong animation?

LEVEL 11 · TODAY'S MISSION

Create a Simple Animation State Plan

YOUR TASK

1. Identify four animation states: Idle, Run, Jump, and Attack.
2. Describe the condition that activates each state.
3. Write simple Python conditional statements to select the state.
4. Create a dictionary that connects each state to its animation frames.
5. Draw a simple State Transition Diagram.

EXPECTED OUTPUT

- A state table.
- A simple Python code example.
- A State Transition Diagram.

OPTIONAL CHALLENGE

What should happen if the player presses Attack while running?

Learning Recap

By the end of this session, you should be able to:

- Explain how multiple frames create animation.
- Understand FPS and animation timing.
- Display animation frames using Python and Pygame.
- Identify Idle, Run, Jump, and Attack states.
- Use conditional statements to select an animation state.
- Connect animation states with their corresponding frames.

Next Step: **Building a more complete State-Machine Animation System using Python and Pygame.**